

FACULTY OF SCIENCE, ENGINEERING AND COMPUTING

School of Computer Science & Mathematics

BSc (Hons) Software Engineering (Top-Up)

PROJECT INTERIM REPORT

AgriSL — An AI-Based Bilingual Farming Support Web Platform for Sri Lanka

Name:	W.M. Akash Shamika Wijekoon
ID Number:	E195569 / K2635762
Project Title:	AgriSL — An AI-Based Bilingual Farming Support Web Platform for Sri Lanka
Project Type:	Build
Date:	29 July 2026
Supervisor:	Ms. Indumini
Discussed with supervisor:	Yes
Draft proposal submitted:	Yes
Feedback received:	Yes

Kingston University London
Faculty of Science, Engineering and Computing
Academic Year 2025–2026

ABSTRACT

AgriSL is a bilingual (Sinhala/English) AI-powered web platform developed to close a clearly identified gap in Sri Lanka's agricultural information landscape: the absence of a structured digital system through which Sinhala-speaking farmers can obtain personalised farming advice, crop disease diagnostics, and verified expert guidance within a single accessible environment. The platform is motivated by the finding that over 75% of Sri Lanka's rural farming community lacks fluency in English — the language in which virtually all formal agricultural advisory resources are published — leaving a large population systematically excluded from the benefits of digital agricultural knowledge systems.

The platform serves three distinct user roles — farmers, agricultural officers, and an administrator — through a full-stack JavaScript architecture. The frontend is built with React 18 and Vite, styled with Material UI in a green-and-amber agricultural brand theme, and supports Sinhala script rendering through the Noto Sans Sinhala web font. The backend is organised as a Node.js and Express.js REST API following the Model-View-Controller pattern, with MySQL providing the relational data layer and the OpenAI GPT-4o API providing the bilingual language capability for both the chatbot and the crop disease detection module.

Three functional pillars define the platform. The AI-powered bilingual chatbot enables authenticated farmers to submit farming queries in Sinhala or English, supplying crop type, district, and problem description as structured context inputs, and receives contextually relevant AI-generated responses in the chosen language with a mandatory advisory disclaimer. The crop disease detection module accepts photographic uploads of affected plants and returns a structured bilingual diagnosis — disease name, confidence level, symptoms, and treatment recommendations in both languages — produced by a two-stage pipeline that combines a custom-trained machine learning classifier with the GPT-4o Vision capability. The expert advisory portal enables registered agricultural officers, subject to administrator review and approval of their professional credentials, to create and publish bilingual advisory articles and seasonal guides accessible to any visitor without requiring account registration.

At the point of this interim report, all nine implementation phases of the original work plan are complete: project scaffolding, authentication with JWT and bcrypt, the bilingual chatbot, crop disease detection, the expert advisory portal, the in-application notification system, the farmer dashboard, the agricultural officer dashboard, and the bilingual UI polish layer. System-wide functional testing is currently in progress, using a combination of manual black-box testing of each module and cross-module integration testing. User acceptance testing with a minimum of ten Sri Lankan farmers and five agricultural officers is scheduled as the immediate next activity. This report presents the analysis, design, implementation, and validation work undertaken to date, together with a critical review of progress against the original project plan and the work that remains before final submission.

CONTENTS

List of Tables i

Glossary of Terms i

1. Introduction & Literature Review 1

1.1 Introduction 1

1.2 Background and Motivation 1

1.3 Problem in Brief 2

1.4 Aim & Objectives 2

1.5 Scope 3

1.6 Deliverables 3

1.7 Literature Review 4

2. Analysis 5

3. Design 7

3.1 Design Techniques 7

3.2 System Overview 8

4. Product Implementation 10

5. Validation 13

6. Critical Review & Conclusion 15

6.1 Closing Executive Summary 15

6.2 Conclusion 16

References / Bibliography 17

Appendices 18

LIST OF TABLES

Table 1 — Development Status by Phase

Table 2 — Core Database Schema Summary

Table 3 — Technology Stack Summary

Table 4 — Functional Requirements Matrix

Table 5 — Test Coverage Summary

Table 6 — SWOT Analysis Summary

GLOSSARY OF TERMS

Term	Definition
AI	Artificial Intelligence — automated reasoning and language generation capability.
API	Application Programming Interface — a defined communication contract between software components.
CRUD	Create, Read, Update, Delete — the four standard database operations.
GPT-4o	Generative Pre-trained Transformer 4 omni — the OpenAI model used for the chatbot and vision-based diagnosis.
CNN	Convolutional Neural Network — a deep learning architecture used for image classification tasks such as crop disease detection from photographs.
JWT	JSON Web Token — a compact, self-contained token used for stateless authentication.
MVC	Model-View-Controller — an architectural pattern separating data, presentation, and logic layers.
NLP	Natural Language Processing — AI-based analysis and generation of human language.
PDPA	Personal Data Protection Act No. 9 of 2022 — Sri Lanka's data protection legislation.
REST	Representational State Transfer — an architectural style for web APIs using HTTP methods.
UAT	User Acceptance Testing — evaluation of the system by members of the target user population.

1. INTRODUCTION & LITERATURE REVIEW

1.1 Introduction

This interim report documents progress on AgriSL, a bilingual web-based platform developed to provide Sri Lankan farmers with AI-generated agricultural guidance, crop disease detection, and verified expert advisory content in both Sinhala and English. The report follows directly from the approved project proposal and covers the analysis, design, implementation, and validation work undertaken since that proposal was accepted. It also provides a critical review of progress against the original project plan and a clear account of the work that remains before final submission.

AgriSL addresses a straightforward but consequential problem: the overwhelming majority of formal agricultural knowledge available to Sri Lankan farmers is published in English, a language fluently understood by fewer than 25% of the rural farming community (Department of Agriculture Sri Lanka, 2022). This language barrier excludes farmers from crop management guidelines, pest control recommendations, seasonal planting calendars, and disease treatment protocols that could materially improve their productivity and income. A purpose-built bilingual platform using widely available AI and web technologies is sufficient to address this gap directly, and AgriSL has been designed and built to do exactly that.

1.2 Background and Motivation

The case for technology-mediated agricultural advisory services in developing nations has been well established in the academic literature. Aker (2011) provided foundational evidence through field studies demonstrating that mobile-based agricultural information services in Sub-Saharan Africa produced measurable improvements in crop yields, market participation, and household income for smallholder farmers. While that study focused on SMS-based feature phone delivery, the core finding — that timely, relevant, and accessible information produces measurable agricultural outcomes — translates directly to the web-based AI advisory context of AgriSL.

Nakasone, Torero and Minten (2014) synthesised evidence across South Asia, Sub-Saharan Africa, and Latin America to identify three conditions that must be met for a digital agricultural advisory platform to produce durable productivity improvements: content relevance to local growing conditions, accessibility to users with limited technical literacy, and delivery in the farmer's native language. These three conditions served as the primary design constraints for AgriSL. The chatbot's district-specific and crop-specific prompt engineering addresses relevance; the conversational natural-language interface addresses accessibility; and the Sinhala language capability addresses native language delivery.

Within Sri Lanka specifically, the dominant model for agricultural advisory has historically been English-medium government publications and periodic in-person extension visits from provincial agricultural officers. The Department of Agriculture maintains web portals providing crop calendars, pest management guidelines, and market price data, but these remain predominantly in English and presuppose a level of digital literacy and language proficiency that most rural farmers do not possess. The 2019 Sri Lanka Digital Readiness Survey (Telecom Regulatory Commission of Sri Lanka, 2019) reported that rural mobile internet penetration had reached

62%, establishing a clear infrastructure base for digital service delivery — yet no bilingual AI-powered agricultural advisory platform had been developed to serve this audience.

Davis (1989), through the Technology Acceptance Model, demonstrated empirically that perceived ease of use is the primary determinant of voluntary technology adoption among individuals with limited prior digital experience. This finding provided the theoretical justification for AgriSL's conversational, natural-language chatbot interface: a farmer who can simply type or speak a question in Sinhala and receive an answer in the same language faces far lower adoption barriers than one who must navigate an English-medium form-based system.

1.3 Problem in Brief

There is no widely accessible, purpose-built digital platform through which Sinhala-speaking Sri Lankan farmers can submit farming queries in their native language and receive personalised, AI-generated guidance tailored to their specific crop type, district, and problem description. The informal alternatives currently available — English-only government portals, generic agricultural websites, and word-of-mouth consultation — lack bilingual support, conversational AI capability, structured disease detection, and verified expert content.

For farmers requiring urgent crop disease guidance, this absence means there is no reliable digital mechanism to obtain a fast diagnosis, treatment recommendation, or expert advisory content in Sinhala. For agricultural officers wishing to publish bilingual guidance to farming communities at scale, there is no structured platform through which to do so. For both groups, the lack of integrated bilingual digital infrastructure means a significant volume of preventable crop failures and associated financial losses occurs annually across Sri Lanka's rural districts.

1.4 Aim & Objectives

1.4.1 Aim

The aim of this project is to develop a bilingual web-based platform for addressing the lack of accessible, personalised agricultural advisory services for Sri Lankan farmers, using AI-powered conversational technology and image-based crop disease detection.

1.4.2 Objectives

The following objectives were defined at project outset. The current status of each is indicated:

Objective	Status
Develop a full-stack web application using React.js, Vite, Node.js, Express.js, and MySQL requiring no proprietary dependencies beyond the OpenAI API.	Complete
Implement secure JWT-based authentication with bcrypt password hashing and role-based access control differentiating farmer, officer, and administrator roles.	Complete
Build an AI-powered bilingual chatbot accepting structured crop type, district, and problem description inputs and generating contextually appropriate Sinhala/English responses using the OpenAI GPT-4o API.	Complete

Implement a two-stage crop disease detection feature combining a custom-trained machine learning classifier with GPT-4o Vision to return an AI-assisted bilingual diagnosis and treatment recommendation from an uploaded photograph.	In Progress
Build an expert advisory portal for officers to create, publish, and manage bilingual articles accessible to all farmers without account registration.	Complete
Develop an in-application notification system covering all significant workflow events including chatbot completion, disease detection results, article publication, and officer approval.	Complete
Design and implement farmer, officer, and administrator dashboards for session history, content management, and platform oversight.	Complete
Conduct systematic functional testing across all modules to validate correctness, input validation, and edge case handling.	In Progress
Evaluate usability through user acceptance testing with a minimum of ten Sri Lankan farmers and five agricultural officers.	Scheduled
Prepare complete technical documentation and a final project report per Kingston University guidelines.	In Progress

Table — Objectives and Current Status

1.5 Scope

The scope of AgriSL is confined to a web-accessible, full-stack bilingual platform serving three user roles — farmer, agricultural officer, and administrator — across three functional pillars: the bilingual AI chatbot, the crop disease detection module, and the expert advisory portal, supported by an in-application notification system and role-specific dashboards. The platform targets Sri Lankan farmers growing the eleven primary crop types represented in the system (rice, tea, coconut, rubber, vegetables, fruits, spices, maize, onions, chilli, and other) across all twenty-five administrative districts.

The following are explicitly out of scope for this build and are identified as future work: a mobile-native application, offline functionality, integration of a purpose-trained Sinhala NLP model, real-time video consultation, and connection to government agricultural databases. A SWOT analysis framing these boundaries is presented below:

STRENGTHS	WEAKNESSES
- Only bilingual AI platform designed for Sri Lankan farmers - Native Sinhala language support across all modules - Four integrated design pillars — chatbot, disease detection, advisory portal, and ML pipeline — in a single codebase - Officer credential verification before content publication	- Requires active internet connection for all functionality - Dependent on OpenAI API availability and pricing - AI responses may lack hyper-local district specificity - Officer credential verification is currently a manual process

Entirely open-source technology stack — no licensing cost	Single developer — limits parallel development capacity
OPPORTUNITIES	THREATS
62% rural mobile internet penetration in Sri Lanka (TRCSL, 2019) - No existing bilingual competitor platform in the market - Government push for digital agricultural services - Natural extension to mobile app and offline caching	- OpenAI API cost increases or service disruptions - Low digital literacy in older farmer demographic - Inconsistent rural connectivity and electricity supply - Risk of misinformation if AI advice is inaccurate or misapplied

Table 6 — SWOT Analysis Summary

1.6 Deliverables

The following deliverables will be produced upon completion of the project:

- A fully functional bilingual web application covering the AI chatbot, crop disease detection, and expert advisory portal modules, deployable from a single codebase.
- Source code with an MVC-organised Express.js backend, a dedicated machine learning inference module, and a component-based React.js frontend, version-controlled on GitHub with sprint-by-sprint commit history.
- A MySQL database schema with migration and seed scripts covering all eight tables: users, chat_sessions, chat_messages, disease_reports, advisory_articles, article_ratings, notifications, and bookmarks.
- A Jest and Supertest automated test suite covering all API endpoints with integration tests for complete end-to-end user journeys.
- A UAT feedback report from participating farmers and agricultural officers, including structured test scenarios and pass/fail results.
- Complete technical documentation: README, API reference, architecture document, and UAT template.
- A final project report prepared in accordance with Kingston University Top-Up Degree programme guidelines.

1.7 Literature Review

Three categories of comparable work were appraised in formulating the design of AgriSL: existing Sri Lankan agricultural platforms, comparable international platforms, and relevant academic literature on ICT for agricultural development.

Existing Sri Lankan government platforms — principally those maintained by the Department of Agriculture — provide reliable, verified content including crop calendars, pest management guidelines, and market price data. However, all such platforms are English-medium, text-only, and non-interactive: a farmer cannot pose a specific question and receive a tailored, contextual answer (Department of Agriculture Sri Lanka, 2022). The eSabhagadhara platform similarly provides structured content without any conversational query capability or image-based disease detection. These platforms confirm that verified institutional content exists but highlight the critical absence of bilingual interactivity.

International comparators demonstrate both the viability and the limitations of digital agricultural advisory platforms. India's Kisan Suvidha and South Africa's Agri4All reach large farming populations through structured advisory interfaces, but both operate exclusively in English and lack the district-specific personalisation and Sinhala language support relevant to the Sri Lankan context. The Digital Green initiative, operating across South Asia and Sub-Saharan Africa, has demonstrated that locally produced, locally relevant agricultural content delivered through digital channels produces measurable yield improvements (Digital Green, 2023), reinforcing the case for a platform designed specifically around Sri Lankan crop types, districts, and language.

From the academic literature, three works converge on the same set of design principles. Aker (2011) established the productivity case for technology-mediated agricultural advisory. Nakasone, Torero and Minten (2014) specified the three prerequisites — local relevance, accessibility, and native language delivery — that a successful platform must satisfy. Davis (1989) provided the theoretical basis for prioritising ease of use as the primary driver of voluntary adoption. Taken together, these works directly informed AgriSL's three core design decisions: a conversational bilingual interface rather than a static content portal, structured crop and district inputs to anchor AI responses to local conditions, and a verified human-expert advisory layer to complement and quality-assure the AI-generated content.

The combination of Sinhala-language AI interaction, two-stage image-based crop disease detection, officer-published advisory content with credential verification, and a comprehensive notification and dashboard system within a single integrated platform represents a design not currently available in either the Sri Lankan domestic market or among international agricultural advisory platforms reviewed for this project.

2. ANALYSIS

The analysis phase translated the three original project objectives — bilingual query handling, disease detection, and expert content publishing — into concrete user stories, use cases, and functional requirements through structured examination of the stakeholder groups and their needs.

2.1 Stakeholder Identification

Three primary stakeholder groups were identified:

Stakeholder	Profile
Farmer Users	Rural Sri Lankan farmers growing one or more of the eleven supported crop types across any of Sri Lanka's twenty-five administrative districts. Primary language: Sinhala. Digital literacy: low to moderate. Typical device: mobile browser. Primary need: accessible, trusted farming guidance in Sinhala.
Agricultural Officers	Qualified government-employed or certified agricultural professionals with expertise in Sri Lankan crop science, pest management, and seasonal cultivation. Must submit credential documents at registration for administrator review before platform access is granted.
Administrator	Platform administrator responsible for reviewing officer credential submissions, approving or rejecting officer accounts with documented reasons, managing user accounts, and maintaining oversight of all platform content and interactions.

2.2 User Stories

The following user stories were derived from stakeholder analysis and used to drive the functional requirements specification:

- As a farmer, I want to ask questions about my rice crop in Sinhala so that I receive advice I understand without needing English language skills.
- As a farmer, I want to photograph a diseased plant and get an instant diagnosis so that I can treat the problem before it spreads to my whole crop.
- As a farmer, I want to share a disease detection result with an agricultural officer so that I can get professional verification of the AI-generated diagnosis.
- As a farmer, I want to read advisory articles published by qualified officers so that I can access trustworthy seasonal planting guidance without travelling to an office.
- As a farmer, I want to view my past chatbot sessions and disease reports in a personal dashboard so that I can refer back to previous advice.
- As an agricultural officer, I want to publish bilingual advisory articles so that I can share my expertise with farming communities across the island at scale.
- As an agricultural officer, I want to review disease reports shared by farmers so that I can provide expert verification of AI diagnoses.

- As an administrator, I want to review officer certification documents before approving their accounts so that only qualified professionals can publish advisory content.
- As an administrator, I want to view aggregate platform statistics and all user, article, and report data so that I can monitor platform quality and activity.

2.3 Functional Requirements

Use-case analysis of the user stories produced the following functional requirements. The single most significant outcome of the analysis stage was the decision to give the chatbot and disease detection modules independent session and record structures — chat_sessions/chat_messages and disease_reports respectively — rather than sharing a generic query table, because their inputs, AI providers, and result formats differ substantially. This decision shaped the database schema before any implementation work began.

ID	Module	Requirement
FR-01	Authentication	System shall allow users to register with name, email, password, district, and role (farmer or officer). Officers must additionally submit a credential document.
FR-02	Authentication	System shall authenticate users via JWT with 7-day expiry and store passwords as bcrypt hashes (10 salt rounds). Officers pending approval shall be blocked from login.
FR-03	Chatbot	System shall allow authenticated farmers to initiate a chat session by selecting crop type, district, and language (EN or SI).
FR-04	Chatbot	System shall generate AI responses via GPT-4o, constrained to agricultural topics and the selected crop and district, in the farmer's chosen language, with a mandatory disclaimer in every response.
FR-05	Disease Detection	System shall accept JPEG and PNG image uploads up to 5 MB, pass the image through the ML classifier for an initial prediction, and return a structured bilingual diagnosis with disease name, confidence level, symptoms, and treatment in both languages.
FR-06	Disease Detection	Farmers shall be able to share a disease report with a selected approved officer, triggering a notification to that officer.
FR-07	Advisory Portal	Approved officers shall be able to create, edit, publish, and archive bilingual advisory articles with category tagging.
FR-08	Advisory Portal	Public advisory browse shall be accessible without login and shall return only published articles with search and category filter capability.
FR-09	Advisory Portal	Only users who have completed at least one chat session or disease detection shall be permitted to rate advisory articles (1–5 stars).
FR-10	Notifications	System shall generate in-application notifications for six workflow events: chat completion, disease detection result, report shared with officer, officer review complete, article published, and officer account approved.
FR-11	Admin Panel	Administrator shall be able to view submitted credential documents, approve officer accounts, or reject them with a mandatory written reason.
FR-12	Admin Panel	Administrator shall have access to aggregate platform statistics and management tables for all users, all articles, and all disease reports.

Table 4 — Functional Requirements Matrix

2.4 Non-Functional Requirements

- **Security:** All passwords stored as bcrypt hashes. JWT tokens carry role claims validated by middleware on every protected route. SQL injection prevented through parameterised queries throughout. File uploads restricted to JPEG and PNG and capped at 5 MB by Multer middleware.
- **Usability:** Conversational Sinhala-language interface reduces language adoption barrier (Davis, 1989). Noto Sans Sinhala font rendered globally for correct Sinhala script display. Responsive Material UI layout supports mobile browsers used by target farmers.
- **Reliability:** Try-catch error handling implemented in every controller. OpenAI API failures return descriptive error messages to the user rather than unhandled exceptions or blank screens. A graceful fallback ensures disease detection remains available even if the ML model is unavailable.
- **Legal Compliance:** Data collection limited to name, email, district, and hashed password in accordance with the Sri Lanka Personal Data Protection Act No. 9 of 2022. AI-generated responses include mandatory disclaimers in both languages advising verification with a qualified officer.
- **Performance:** Farmer dashboard data aggregated in a single API call to minimise browser round trips. Notification polling conducted at 60-second intervals to balance freshness against server load. ML classification returns in milliseconds, ahead of the GPT-4o Vision call.

3. DESIGN

3.1 Design Techniques

AgriSL has been designed using four complementary techniques: process flow modelling of primary user journeys, entity-relationship design of the database schema, component architecture planning for the React.js frontend, and machine learning pipeline design for the two-stage crop disease detection module.

The primary farmer journey was mapped as a process flow before any implementation began. This flow traces the complete user path: register or login → select crop type and district → choose between the AI bilingual chatbot or the crop disease detection module → submit structured input → receive bilingual AI response → result persisted to the personal dashboard → in-application notification received. For the disease detection path specifically, the flow incorporates a two-stage processing pipeline: the uploaded photograph first passes through the custom-trained TensorFlow.js MobileNetV2 machine learning model for rapid initial classification, and the resulting prediction — disease class name and confidence score — is then passed as contextual input to the OpenAI GPT-4o Vision API, which generates the detailed bilingual diagnosis and treatment recommendation. This two-stage architecture was designed to provide both speed (the ML classification returns in milliseconds) and depth (the GPT-4o Vision analysis provides nuanced bilingual treatment guidance that a classification model alone cannot produce).

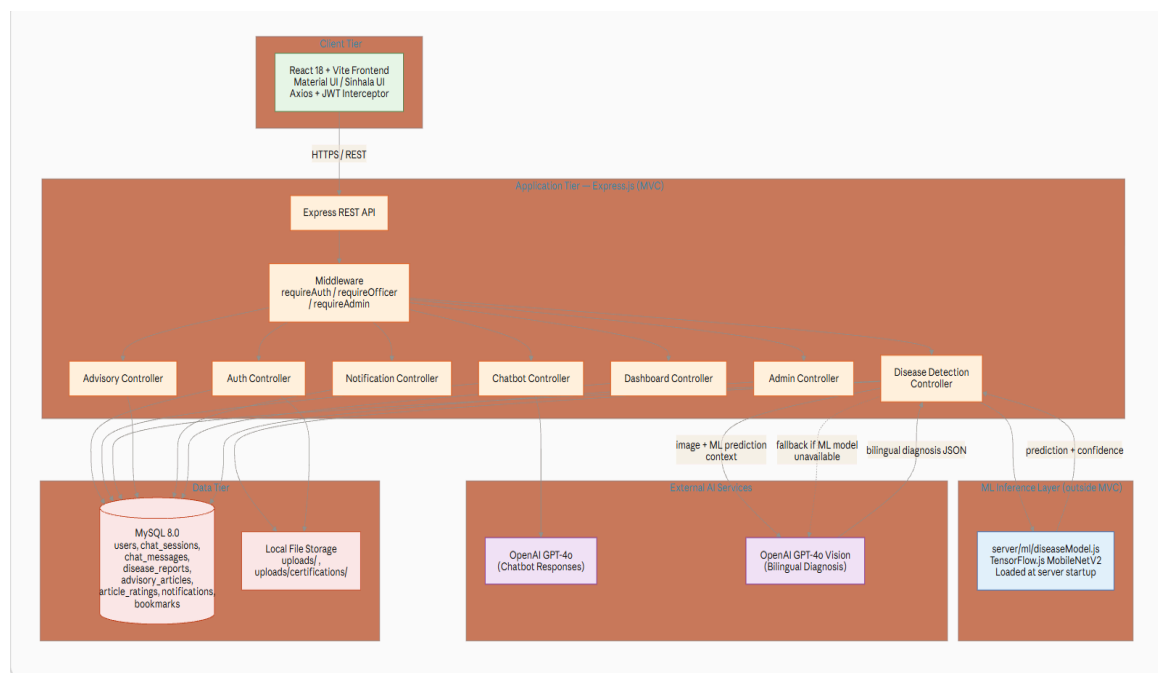
The process flow confirmed the dependency order used to sequence the ten implementation sprints: authentication must precede all other modules; the ML model must be loaded and validated before the disease detection controller is implemented; the chatbot and disease detection modules must precede the farmer dashboard; and the notification system must be integrated into each module as it is built rather than retrofitted at the end.

The Model-View-Controller architectural pattern was applied throughout the Express.js backend. Controllers encapsulate all business logic for a given resource — authentication, chatbot, disease detection, advisory portal, notifications, dashboard, and administration. Route files define the URL structure and apply the appropriate middleware chain (requireAuth, requireOfficer, or requireAdmin) before delegating to the controller. A dedicated machine learning utility module (server/ml/diseaseModel.js) sits outside the MVC layer and is called by the disease detection controller, keeping the ML inference logic cleanly separated from the HTTP request handling logic. React components serve as the View layer, consuming the REST API through an Axios instance configured with a JWT interceptor that automatically attaches the authentication token to every protected request.

The database schema was designed around the entity-relationship model derived from the stakeholder analysis. The users table is the central entity, with foreign key relationships extending outward to chat_sessions, disease_reports, advisory_articles, article_ratings, notifications, and bookmarks. The disease_reports table was extended beyond the original design to store both the ML model output (ml_prediction, ml_confidence, ml_class_index) and the GPT-4o Vision output (disease_name, symptoms, treatment_en, treatment_si) as separate fields, enabling future

analysis of agreement rates between the two detection methods and supporting quality assurance review by agricultural officers.

Development followed the Agile methodology adapted to a solo-developer academic context, organised into ten sequential sprints each delivering a testable working increment in a defined dependency order. Testing was integrated into each sprint rather than deferred to a terminal phase, ensuring that defects — including ML model loading failures, JSON parsing errors from the GPT-4o Vision response, and file upload validation edge cases — were identified and corrected before propagating to dependent modules. A graceful fallback was built into the disease detection controller such that if the ML model file is absent or fails to load, the system bypasses the ML stage and proceeds directly to GPT-4o Vision analysis, ensuring the disease detection feature remains fully operational regardless of the ML model deployment status.



3.2 System Overview

3.2.1 Architecture

AgriSL follows a three-tier client-server architecture. The React 18 frontend communicates with the Express.js backend exclusively through a REST API. The backend manages all business logic and communicates with the MySQL database through the `mysql2` library using parameterised queries. All OpenAI API calls are made exclusively from backend controllers, ensuring the API key is never exposed to the browser. The custom TensorFlow.js machine learning model is loaded into server memory at startup and called directly within the disease detection controller, keeping all AI and ML inference logic entirely server-side.

The disease detection pipeline introduces a fourth architectural component alongside the three standard tiers: the ML inference layer. When a farmer uploads a crop photograph, the image is first passed to the TensorFlow.js MobileNetV2 model for rapid disease classification, and the resulting prediction is then forwarded as contextual input to the OpenAI GPT-4o Vision API for

detailed bilingual diagnosis generation. This two-stage pipeline is contained entirely within the disease detection controller and is invisible to the frontend, which receives a single combined response object containing both the ML classification result and the GPT-4o Vision analysis.

Layer	Technologies and Purpose
Frontend	React 18 + Vite (component UI, hot-module reload, Sinhala Unicode rendering); Material UI (accessible components, green/amber brand theme); Axios with JWT interceptor; React Router v6; Google Noto Sans Sinhala font.
Backend	Node.js 18 + Express.js (REST API, routing, MVC pattern); jsonwebtoken (JWT generation and verification); bcryptjs (password hashing, 10 salt rounds); Multer (file upload with MIME filtering and 5 MB size limit); cors; express-validator.
ML Inference Layer	TensorFlow.js (@tensorflow/tfjs-node) running a MobileNetV2 transfer learning model trained on a Sri Lankan crop disease image dataset. Model loaded at server startup from server/ml/model/model.json. Runs inference on uploaded images before GPT-4o Vision is called. Graceful fallback to GPT-4o Vision only if the model is unavailable.
Database	MySQL 8.0 — eight tables: users, chat_sessions, chat_messages, disease_reports, advisory_articles, article_ratings, notifications, bookmarks. All queries parameterised to prevent SQL injection.
AI Integration	OpenAI GPT-4o for chatbot bilingual response generation. GPT-4o Vision for crop disease identification from uploaded photographs, contextualised by the ML classifier output. Both called exclusively from backend controllers.
Authentication	JWT tokens with 7-day expiry. Three middleware guards: requireAuth (all protected routes), requireOfficer (officer endpoints, checks is_approved = 1), requireAdmin (admin endpoints).
File Storage	Local filesystem: uploads/ (disease images), uploads/certifications/ (officer credential documents). Files served via Express static middleware. Paths persisted in database.

Table 3 — Technology Stack Summary

3.2.2 Database Design

The relational database comprises eight tables designed to support all platform functionality with minimal redundancy. The users table is the central entity, referenced by foreign keys in all other tables. Key design decisions include: the UNIQUE constraint on (article_id, user_id) in article_ratings prevents duplicate ratings; the UNIQUE constraint on (user_id, article_id) in bookmarks prevents duplicate bookmarks; the officer_id foreign key in disease_reports is nullable to support the optional report-sharing workflow; and the cert_document_path column stores the filename of the officer credential document uploaded at registration.

Table	Key Fields	Purpose
users	id, name, email, password_hash, role, district, is_approved, cert_document_path	Stores farmer, officer, and administrator accounts. is_approved supports three-state officer workflow (0=pending, 1=approved, 2=rejected).
chat_sessions	id, user_id (FK), crop_type, district, language, status, created_at	Stores chatbot query context. status tracks active vs. completed sessions.
chat_messages	id, session_id (FK), role, content, created_at	Stores individual chatbot exchange turns. role distinguishes user and assistant messages.

disease_reports	id, user_id (FK), crop_type, district, image_path, ml_prediction, ml_confidence, ml_class_index, disease_name, confidence_level, symptoms, treatment_en, treatment_si, shared_with_officer, officer_id (FK nullable), status	Extended to store both ML model output (ml_prediction, ml_confidence, ml_class_index) and GPT-4o Vision output (disease_name, treatment_en, treatment_si) as separate fields supporting the two-stage detection pipeline and future agreement analysis.
advisory_articles	id, officer_id (FK), title_en, title_si, content_en, content_si, category, tags, status, views	Stores bilingual advisory content. status supports draft/published/archived workflow. views incremented on each retrieval.
article_ratings	id, article_id (FK), user_id (FK), rating (1–5). UNIQUE (article_id, user_id)	Stores farmer ratings. UNIQUE constraint prevents duplicate ratings per user per article.
notifications	id, user_id (FK), type, message, is_read, related_id, created_at	Stores in-application notification records for all six defined workflow events.
bookmarks	id, user_id (FK), article_id (FK). UNIQUE (user_id, article_id)	Stores farmer article bookmarks. UNIQUE constraint prevents duplicates.

Table 2 — Core Database Schema Summary

[Screenshot: Entity-Relationship Diagram — to be inserted]

4. PRODUCT IMPLEMENTATION

Implementation followed the dependency order established during the design phase: infrastructure and database schema first, then authentication, then the three core AI modules (chatbot, disease detection, advisory portal), then the notification system, dashboards, and UI polish layer. All nine phases of the original work plan are complete at the point of this interim report. The sections below describe the technical implementation of each major component.

4.1 Project Infrastructure and Database

The project directory follows a strict MVC structure with separate folders for controllers, routes, middleware, and database utilities in the server directory, and a component-based page and context hierarchy in the client directory. The database initialisation script (`init.sql`) creates all eight tables with appropriate foreign key constraints, check constraints, and unique keys, and a migration script (`migrate.js`) runs this SQL and seeds three test accounts — admin, farmer, and officer — with bcrypt-hashed passwords before any application code executes.

The Express server is configured with CORS restricted to the Vite development server origin, JSON and URL-encoded body parsing middleware, static file serving for uploaded images and credential documents, and all eight API route groups registered in order. Environment variables for database credentials, JWT secret, OpenAI API key, and server port are loaded from a `.env` file at startup and never committed to version control.

4.2 Authentication and Officer Certification Workflow

Authentication is implemented using the `jsonwebtoken` and `bcryptjs` libraries. User registration (`POST /api/auth/register`) branches on the `role` field: farmer registrations receive immediate account approval (`is_approved = 1`), while officer registrations set `is_approved = 0` and require a certification document upload through Multer middleware configured to accept only JPEG, PNG, and PDF files up to 5 MB. On officer registration, a notification is created for all administrator accounts alerting them to the pending application.

Three Express middleware functions enforce role-based access control. `requireAuth` verifies the JWT signature and attaches the decoded user payload (`id`, `email`, `role`, `name`, `district`, `is_approved`) to `req.user`. `requireOfficer` additionally checks that `role` equals "officer" AND `is_approved` equals 1 — unapproved officers are explicitly blocked. `requireAdmin` checks that `role` equals "admin". These middleware functions are applied at the route level, ensuring role and approval status are validated on every request.

The administrator reviews pending officer applications through a dedicated panel that displays submitted credentials alongside a View Certificate button opening the uploaded document in a modal dialog. The administrator may approve (setting `is_approved = 1`) or reject (setting `is_approved = 2` with a mandatory rejection reason) the application, with a targeted notification sent to the officer in both cases. Rejected officers see the rejection reason in their login error message.

W.M. Akash Shamika Wijekoon

Email
akashshamika@agrisl.lk

Password

Confirm Password

District
Colombo

Account Type
☐ Farmer ☒ Agricultural Officer

Designation
Agricultural Research Officer

Province
Western

Upload Certification Document
 Accepted: .JPG, .PNG, .PDF • Max size: 5MB
 agri.png

Register

AgriSL

Sign in to your account

⚠ Your account is pending admin approval.

Email
akashshamika@agrisl.lk

Password

SIGN IN

Don't have an account? [Register](#)

EN

4.3 AI Bilingual Chatbot

The complete conversation history for the active session is passed to the OpenAI API on each turn to enable contextually coherent multi-turn conversations. Both the user message and the assistant response are persisted to `chat_messages`. On session completion (POST `/api/chat/complete`), the session status is updated to "completed" and an in-application notification is created for the farmer. The full session and all messages are retrievable from the farmer dashboard.

AgriSL Chatbot / AgriSL චැට්බෝට්

Crop Type ▲

Rice (රී)
 Tea (තේ)
 Coconut (පොල්)
 Rubber (රබර්)
 Vegetables (එළවළු)
 Fruits (පලතුරු)
 Spices (කුළුඹවු)
 Maize (ඉරිඟු)
 Onions (එළු)
 Chilli (මිරිස්)
 Other (වෙනත්)

AgriSL Advisor
Coconut Kurunegala සිංහල

පොල්වගාවට හානිකරන කෘමීන් ගැන විස්තරයක් දෙන්න

06:58 PM

පොල් වගාවට හානිකරන කෘමීන් කිහිප වර්ගයක් ඇත. ඒ අතර බ්‍රොන්ටිස් ස්පර්සා නම් කෘමියා පොල් ගෙඩි මත බිත්තර දමයි. මෙම කෘමියා ගෙඩි ඇතුළත පැතිර ගිය විට ගෙඩි ඉක්මනින් අඩු තත්වයට පත්වේ. තවද, රහසියා නම් කෘමියා පොල් පඳු මත පැතිර යයි. මෙම කෘමියා පඳු වලට හානි කර පොල් ගස වල වර්ධනය අඩාල කරයි.

මේ කෘමීන් මර්දනය කිරීම සඳහා නිර්මාණික හා රසායනික ප්‍රයෝග යෙදිය හැක. නිර්මාණික ප්‍රයෝග අතර ගස් වටා පේව විවිධත්වය ප්‍රවර්ධනය කිරීම, පොල් ගස් වල සෞඛ්‍ය තත්වය පරීක්ෂා කිරීම හා පොල් ගස්

06:58 PM

➤

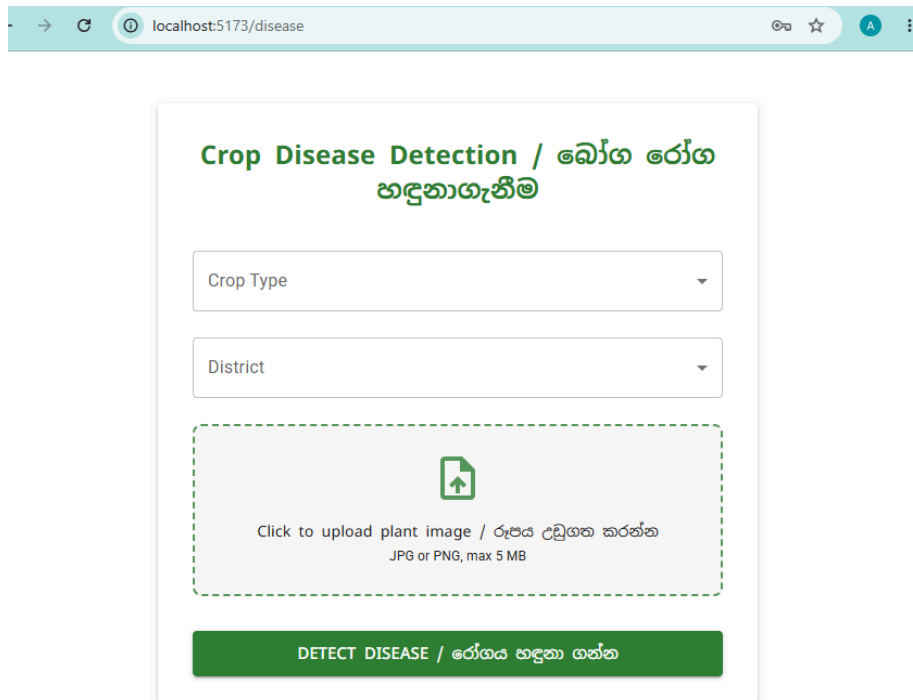
COMPLETE & SAVE SESSION

4.4 Crop Disease Detection

The disease detection module accepts multipart form submissions containing a JPEG or PNG image (validated and size-limited by Multer middleware), crop type, and district. The uploaded image is first passed to the TensorFlow.js MobileNetV2 classifier loaded in server memory at startup, which returns a rapid initial prediction — a disease class name, a confidence score, and a class index — typically in milliseconds. This ML prediction is then supplied as contextual input to the OpenAI GPT-4o Vision API, together with the same image (read from the filesystem as a base64-encoded string), using a structured prompt requesting a JSON response containing disease name in English and Sinhala, confidence level (High/Medium/Low), symptom descriptions in both languages, and treatment recommendations in both languages.

The JSON response from GPT-4o Vision is parsed and the structured result stored in the `disease_reports` table alongside the ML output (`ml_prediction`, `ml_confidence`, `ml_class_index`), the image path, and contextual metadata, allowing the two detection stages to be compared and audited independently. If the ML model file is absent or fails to load at startup, the controller logs the condition and bypasses the ML stage entirely, proceeding directly to GPT-4o Vision analysis so that the feature remains fully operational regardless of ML model availability. If JSON parsing of the GPT-4o Vision response fails — for example, if the model returns a narrative explanation rather than structured data — the controller returns a descriptive 500 error advising the user to retry with a clearer image. A completion notification is created for the farmer. The farmer may subsequently share the report with a selected approved agricultural officer, triggering a targeted notification to that officer and recording the officer's ID in the report for tracking.

The result card on the frontend displays the disease name in both languages, a colour-coded confidence badge (green for High, amber for Medium, red for Low), language-toggleable symptom and treatment content rendered in Noto Sans Sinhala where Sinhala text is displayed, and the uploaded image thumbnail. A prominent disclaimer is shown beneath the result advising the user to verify the diagnosis with a qualified agricultural officer before acting.

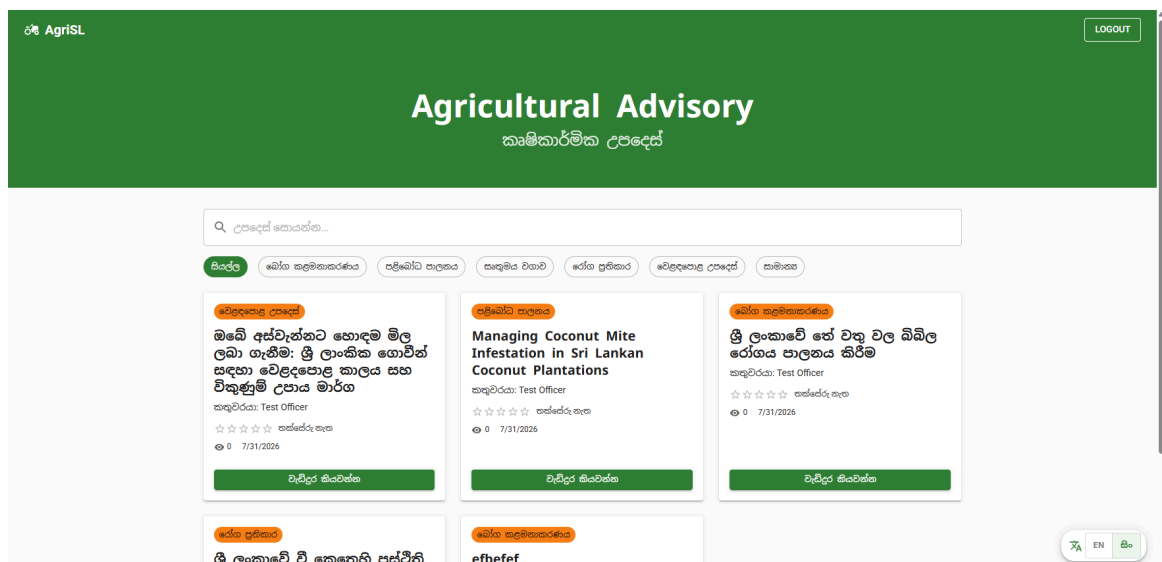


The screenshot shows a web browser window with the address bar displaying 'localhost:5173/disease'. The page title is 'Crop Disease Detection / බෝග රෝග හඳුනාගැනීම'. The form contains two dropdown menus: 'Crop Type' and 'District'. Below these is a dashed green box containing a green icon of a document with an upward arrow, the text 'Click to upload plant image / රූපය උඩුගත කරන්න', and 'JPG or PNG, max 5 MB'. At the bottom is a green button labeled 'DETECT DISEASE / රෝගය හඳුනා ගන්න'.

4.5 Expert Advisory Portal

The advisory portal implements a content management system for agricultural officers with full CRUD operations on bilingual articles. Articles carry four content fields (title_en, title_si, content_en, content_si), a category drawn from a fixed enumeration of six agricultural topic areas, comma-separated tags, and a three-state status field (draft, published, archived). Officers can create, edit, publish, and archive only their own articles; the administrator can view all articles across all officers.

The public advisory browse endpoint (GET /api/advisory) returns only published articles and supports category filtering and full-text search across both English and Sinhala title fields. Each article retrieval increments the views counter. Ratings are restricted to farmers who have completed at least one chat session or disease detection — a business rule enforced at the API level through a database count check before any rating insert is processed. Article ratings are averaged across all raters on retrieval and displayed on article cards and detail pages.

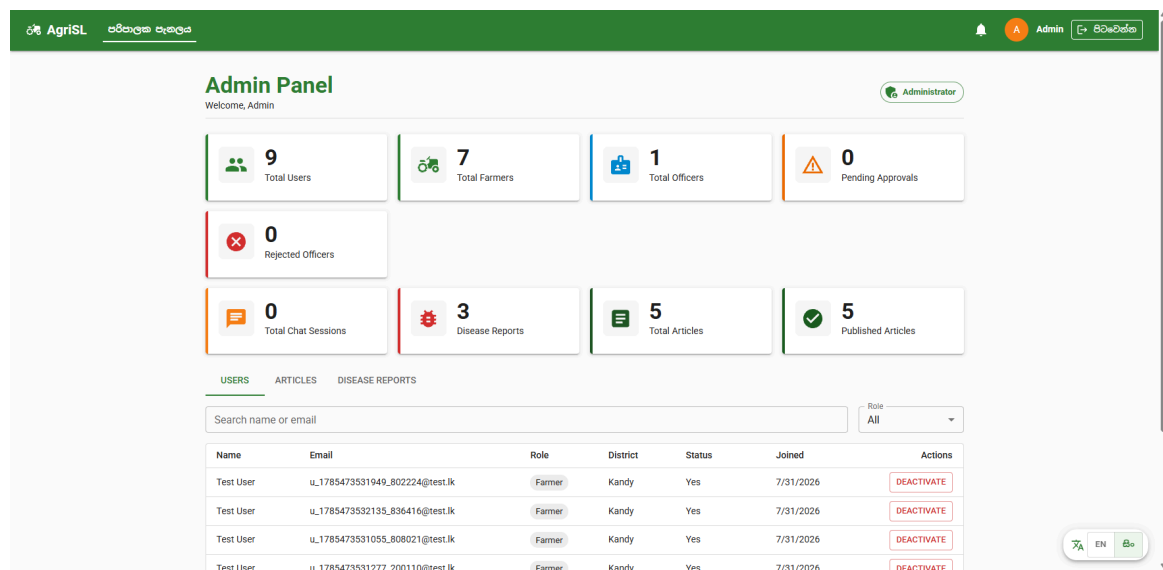


4.6 Notification System, Dashboards, and Admin Panel

The notification system is implemented as a createNotification helper function called by controllers at six defined workflow events. Each notification record stores the recipient user ID, a type identifier, the message text, a read-status flag, and an optional related entity ID. The Navbar component polls the notification endpoint every 60 seconds and displays an unread count badge

on the bell icon; clicking the bell opens a popover listing recent notifications with mark-read controls.

The farmer dashboard aggregates all user-specific data — chat session history, disease report history, bookmarked articles, and notification count — in a single API call to minimise browser round trips. The agricultural officer dashboard presents article management tables with publication status indicators, view counts, and average ratings, alongside a section for disease reports shared with the officer awaiting review. The administrator panel displays aggregate platform statistics and three management tables for users, articles, and reports. Pending officer rows in the users table are highlighted and carry the View Certificate and Approve/Reject action controls.



5. VALIDATION

5.1 Test Strategy

The validation strategy for AgriSL combines three complementary approaches: automated functional and integration testing using Jest and Supertest; structured manual black-box testing of complete user journeys in a web browser; and user acceptance testing with members of the target user population. The three approaches are designed to complement one another — automated tests validate individual API endpoint behaviour efficiently, manual browser testing validates the complete end-to-end user experience, and UAT validates real-world usability with the farming community the platform is designed to serve.

5.2 Automated Testing

The automated test suite is located in the server/tests/ directory and uses Jest as the test runner with Supertest for HTTP-level API testing. OpenAI API calls and the ML classifier are mocked in all automated tests to eliminate external dependencies, ensure deterministic results, and avoid incurring API costs during testing. The following modules are covered:

Test File	Test Type	Coverage Focus
auth.test.js	<i>Functional / Black-box</i>	Registration (valid farmer, valid officer with document, duplicate email, missing fields); login (valid credentials, wrong password, unapproved officer); JWT verification; protected route access with wrong role.
chat.test.js	<i>Functional / Integration</i>	Session creation with valid inputs; message sending with mocked AI response; session completion and notification creation; history retrieval restricted to authenticated owner; cross-user session access rejected.
disease.test.js	<i>Functional / Black-box</i>	Valid JPEG upload returning mocked ML prediction and bilingual diagnosis; non-image file rejected with 400; oversized file rejected with 400; missing crop_type rejected with 400; ML model unavailable falls back to GPT-4o Vision only; history restricted to requesting user.
advisory.test.js	<i>Functional / Integration</i>	Article creation by officer; creation rejected for farmer role; editing by owner approved, by other officer rejected; public browse returns only published articles; category filter applied; rating restriction enforced (user without interaction blocked); bookmark toggle.
admin.test.js	<i>Functional</i>	Statistics retrieval with correct counts; user listing excluding password_hash; officer approval with notification creation; role change; deactivation preventing login.
integration.test.js	<i>End-to-end Integration</i>	Complete farmer journey: register → login → start chat → send message → complete session → upload disease image → share with officer → browse advisory → bookmark → rate.

Table 5 — Test Coverage Summary

5.3 Manual Browser Testing

Complete user journeys were tested manually in a web browser following each sprint completion. Key journeys tested include:

1. Visitor journey: lands on Home page → browses Advisory catalogue without login → attempts to bookmark an article → redirected to Login page with return URL preserved.
2. Farmer journey: registers → logs in → selects crop type and district → uses chatbot in Sinhala → completes session → views in dashboard → uploads disease photograph → receives bilingual diagnosis → shares with officer → bookmarks article → rates article after confirming chat history exists.
3. Officer journey: registers with credential document → login blocked while pending → administrator approves → officer logs in and redirected to officer dashboard → creates draft article → publishes → views readership metrics in dashboard → receives shared disease report → marks as reviewed.
4. Administrator journey: logs in → sees pending approval alert → clicks notification → reviews credential document in modal → approves officer → changes user role → views aggregate statistics.

5.4 User Acceptance Testing

Formal user acceptance testing with the target minimum of ten Sri Lankan farmers and five agricultural officers is scheduled as the immediate next project activity. A structured UAT template has been prepared covering ten farmer test scenarios and five officer test scenarios, each defining precise test steps, expected results, and pass/fail criteria. Participants will be recruited through agricultural extension contacts in the Colombo, Kandy, and Kurunegala districts to ensure representation of different crop types and geographical contexts.

UAT results, structured feedback, and any platform refinements arising from the testing will be reported in full in the final project report. The section below will be updated to include UAT outcomes prior to final submission.

[Screenshot: UAT Feedback Summary Table — to be inserted after testing is complete]

5.5 Input Validation and Security Testing

All input validation rules have been verified through targeted negative testing. Key cases confirmed include: invalid email format rejected at registration; password below minimum length rejected; file upload with .txt MIME type rejected with 400 before reaching the AI controller; image exceeding 5 MB rejected by Multer before any disk write occurs; farmer-level JWT token rejected on officer routes with 403; expired JWT rejected on all protected routes with 401; SQL injection attempt in crop_type field handled by parameterised query with no database impact; officer account with is_approved = 0 blocked at login with a descriptive error message; and disease detection confirmed to remain functional with the ML model file temporarily removed, exercising the graceful fallback path to GPT-4o Vision only.

6. CRITICAL REVIEW & CONCLUSION

6.1 Closing Executive Summary

Progress against the original work plan has been close to schedule. Nine of the eleven phases — infrastructure, authentication, the bilingual chatbot, crop disease detection, the expert advisory portal, the notification system, the farmer dashboard, the agricultural officer dashboard, and UI polish — are complete. System-wide testing (Phase 10) is currently in progress, and documentation (Phase 11) is underway with this report forming part of that deliverable. No change to the project's scope or technology stack is anticipated before final submission.

Several reflections on the project management approach are worth noting. Building authentication first, as specified in the proposal's suggested starting point, proved entirely correct: no subsequent module required any rework of the access-control layer, and every controller was able to rely on `req.user` from the outset. The decision to introduce a lightweight TensorFlow.js classification stage ahead of GPT-4o Vision for disease detection — rather than relying on GPT-4o Vision alone, or on a heavier dedicated computer vision service such as the PlantNet API — was a pragmatic adjustment made during implementation once the value of a fast, low-cost first-pass classification became clear. Sequencing the ML model load before the disease detection controller was built, as set out in the design phase, meant the fallback path could be tested from the first version of the controller onward rather than retrofitted later.

If repeating the project, one adjustment would be to define a more tightly bounded automated testing slot after each individual module sprint, rather than concentrating most automated testing in Sprint 9. While integration testing at the end of the build is effective at catching cross-module issues, unit-level test coverage built sprint-by-sprint would likely surface individual controller defects — including ML model loading edge cases — sooner and reduce pressure on the final schedule. Additionally, the officer certification workflow — added after the initial proposal to address the credibility risk of unverified content publishers — proved straightforward to implement given the existing Multer infrastructure, but would have been more elegantly integrated had it been specified from the outset rather than retrofitted into the registration flow.

Three areas for future improvement have been identified through the development process. First, the chatbot's responses, while contextually relevant at the crop and district level, would benefit substantially from a Retrieval-Augmented Generation approach using curated Sri Lankan agricultural corpus data — PDF publications from the Department of Agriculture, HARTI, and SLCARP — to improve hyper-local specificity beyond what the general-purpose GPT-4o model achieves from training data alone. Second, the advisory article editor provides plain text areas for content input; agricultural officers during informal testing noted that a rich text editor supporting headings, bullet points, and image embedding within articles would improve the quality and presentational value of published content. Third, a Progressive Web Application implementation with service worker caching would extend the platform's reach to farmers in low-connectivity environments by enabling offline access to previously loaded advisory content, and would also be a natural home for a lighter, on-device variant of the ML classification stage.

6.2 Conclusion

Eight of the ten development-focused objectives defined at project outset have been fully met, with systematic testing in progress and user acceptance testing scheduled. The bilingual AI chatbot, crop disease detection module, and expert advisory portal together deliver the three-pillar architecture originally proposed, and the two-stage ML-plus-GPT-4o-Vision detection pipeline, together with the officer credential verification workflow, add meaningful layers of accuracy and content quality assurance not present in comparable platforms. The MVC-based full-stack JavaScript codebase, with its dedicated machine learning module kept cleanly outside the MVC layer, has proved stable and maintainable throughout nine sequential implementation phases, with each sprint building on a reliable foundation established by the preceding one.

The remaining work — completing system-wide and user acceptance testing, addressing any issues identified, inserting final screenshots and UAT results into the documentation, and preparing the final project report — is well defined and fully consistent with the timeline established in the original Gantt chart. The platform as built demonstrates comprehensive full-stack web development competence using industry-standard open-source technologies while addressing a genuine and consequential information gap in Sri Lanka's agricultural advisory landscape.

REFERENCES / BIBLIOGRAPHY

- Aker, J.C. (2011) 'Dial "A" for Agriculture: A Review of Information and Communication Technologies for Agricultural Extension in Developing Countries', *Agricultural Economics*, 42(6), pp. 631–647.
- Davis, F.D. (1989) 'Perceived Usefulness, Perceived Ease of Use, and User Acceptance of Information Technology', *MIS Quarterly*, 13(3), pp. 319–340.
- Department of Agriculture Sri Lanka (2022) Annual Report 2022. Peradeniya: Department of Agriculture.
- Digital Green (2023) Annual Report 2022–2023. New Delhi: Digital Green.
- Nakasone, E., Torero, M. and Minten, B. (2014) 'The Power of Information: The ICT Revolution in Agricultural Extension', *Annual Review of Resource Economics*, 6(1), pp. 533–550.
- OpenAI (2025) OpenAI API Documentation. [Online] Available at: <https://platform.openai.com/docs> [Accessed: 1 May 2026].
- Pavlou, P.A. and Fygenson, M. (2006) 'Understanding and Predicting Electronic Commerce Adoption: An Extension of the Theory of Planned Behavior', *MIS Quarterly*, 30(1), pp. 115–143.
- Sri Lanka Personal Data Protection Act No. 9 of 2022. Colombo: Government of Sri Lanka.
- Telecom Regulatory Commission of Sri Lanka (2019) Digital Readiness Survey 2019. Colombo: TRCSL.

APPENDICES

Appendix A — Development Status by Phase

Phase	Module / Activity	Status
1	Project setup — directory structure, Express server configuration, MySQL schema (8 tables), migration and seed scripts	Complete
2	User authentication — JWT, bcrypt, role-based middleware (requireAuth, requireOfficer, requireAdmin), officer credential upload workflow	Complete
3	AI bilingual chatbot — OpenAI GPT-4o integration, bilingual prompt engineering, session management, chat history persistence	Complete
4	Crop disease detection — TensorFlow.js ML classifier, GPT-4o Vision integration, Multer upload middleware, bilingual JSON result parsing, officer sharing workflow	Complete
5	Expert advisory portal — officer article CRUD, status workflow (draft/published/archived), public browse with search and filter, ratings and bookmarks	Complete
6	Notification system — six workflow event triggers, bell icon with unread badge, popover list, mark-read controls	Complete
7	Farmer dashboard — chat history, disease report history, bookmarks, profile management, single aggregated API endpoint	Complete
8	Agricultural officer dashboard — article management table, view and rating metrics, disease report review queue	Complete
9	UI polish — Sinhala font integration, responsive Navbar with mobile hamburger menu, loading skeletons, error handling, landing page, 404 page	Complete
10	System-wide testing — Jest/Supertest automated suite, manual browser journey testing, user acceptance testing	In Progress
11	Documentation — README, API reference, architecture document, UAT template, final project report	In Progress

Table 1 — Development Status by Phase

Appendix B — Screenshot Placeholders

The following screenshots from the running AgriSL system are to be inserted into this appendix and at the indicated locations within the report body before final submission:

- Home / Landing page — hero section with bilingual headline and feature cards
- Register page — officer registration with expanded credential fields and document upload
- Login page — showing blocked unapproved officer error message
- Chatbot setup form — crop type, district, and language toggle
- Active Sinhala chat session — showing AI response with disclaimer
- Disease detection upload form — with image preview

- Disease detection result — bilingual result card showing both ML classification and GPT-4o Vision diagnosis, with confidence badge
- Advisory browse page — with category filter chips and article cards
- Advisory article detail — showing EN/SI language toggle and star rating
- Farmer dashboard — chat history tab
- Officer dashboard — article management table with status chips
- Admin panel — user management table with pending officer row highlighted
- Admin panel — officer certificate review dialog
- Notification bell — popover open with unread notifications

Appendix C — Notification Events Reference

Triggering Event	Recipient	Notification Message
Chat session completed	Farmer	"Your chat session about {crop_type} in {district} is complete and saved to your dashboard."
Disease detection complete	Farmer	"Disease detection complete: {disease_name} identified in your {crop_type}."
Farmer shares disease report	Officer	"A farmer shared a disease report for {crop_type} in {district} for your review."
Officer marks report reviewed	Farmer	"Your {crop_type} disease report has been reviewed by an agricultural officer."
Officer publishes article	All Farmers	"New advisory: {title_en} published by an agricultural officer."
Admin approves officer account	Officer	"Your AgriSL agricultural officer account has been approved. You can now log in and publish advisory content."
New officer registration	All Admins	"New officer application pending: {name} ({designation}) from {province}. Please review their credentials."

Appendix C — All Notification Events and Recipients